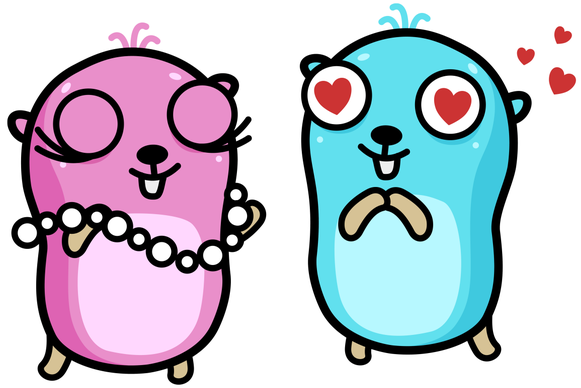
Destilando o Green-Tea

O novo GC do Go

Matheus Mina















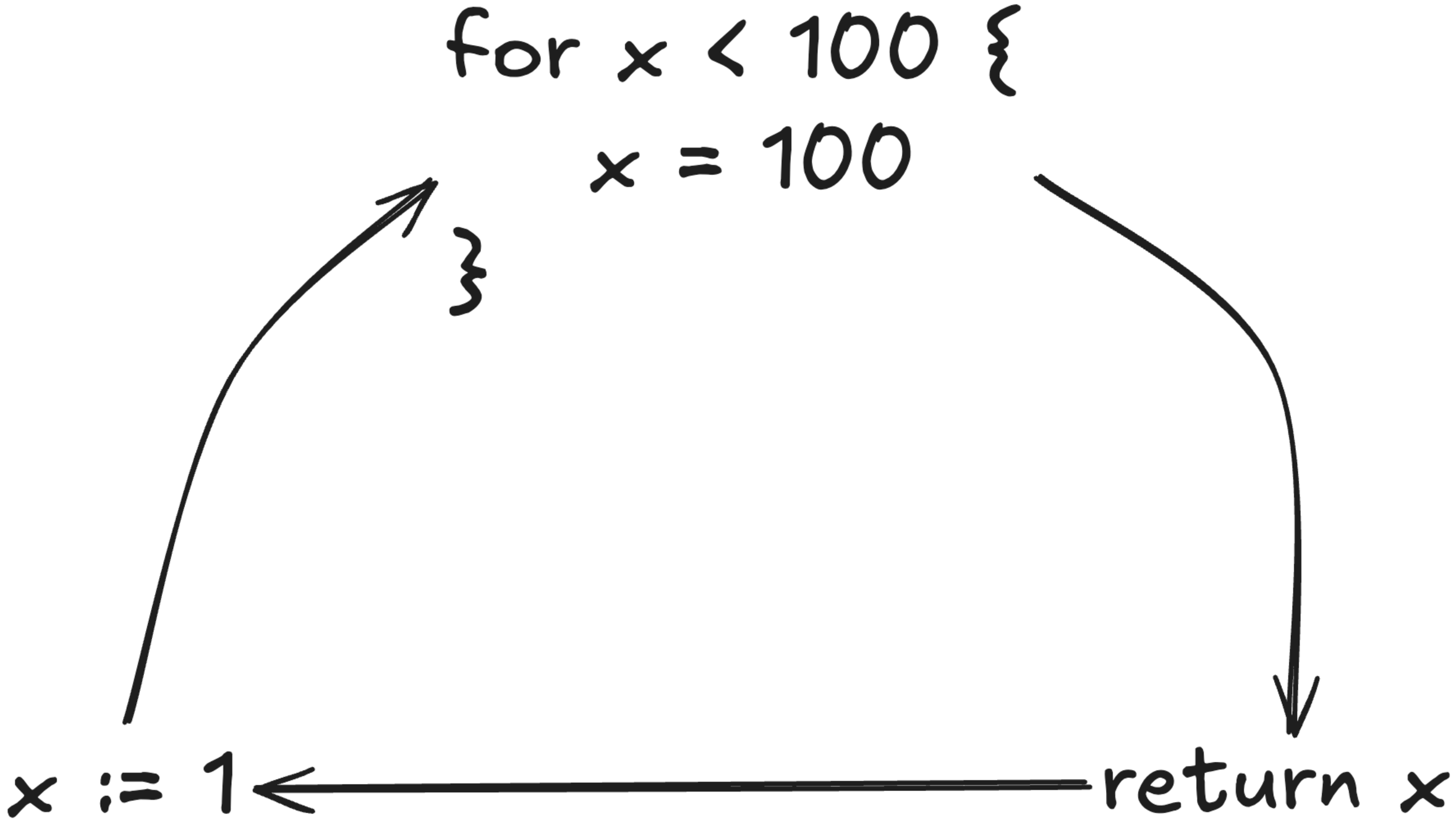


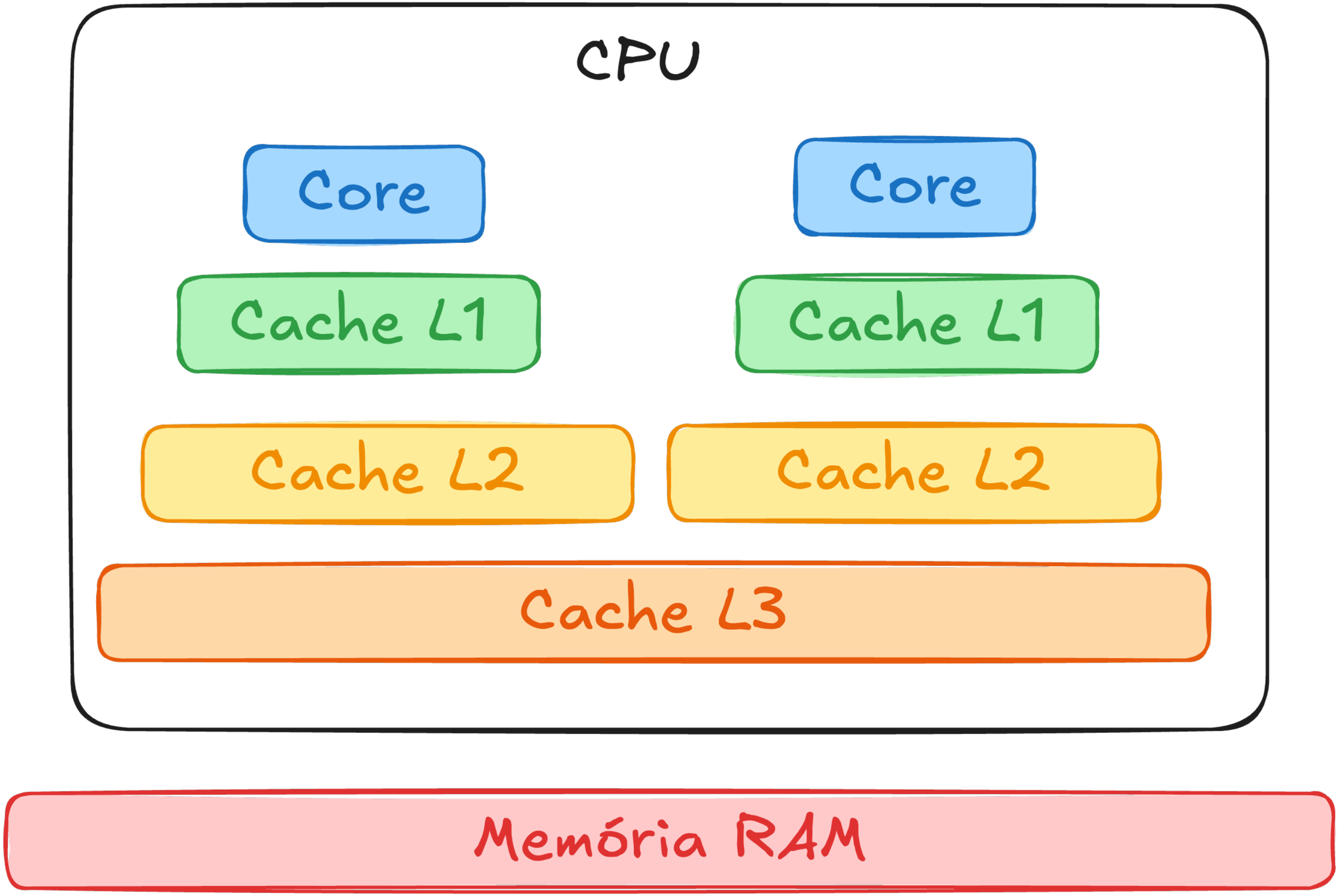


O que é um GC?






for x < 100 {
x = 100
}
x := 1
return x


CPU
Core
Core
Cache L1
Cache L1
Cache L2
Cache L2
Cache L3
Memória RAM

Por que a memória não acaba?



Garbage collector!



Quais são os algoritmos mais famosos?



- Tracing GC
  - Mark and Sweep, Stop and Copy
- Generational GC
  - G1, ZGC, Shenandoah
- Reference Counting
  - ARC



Ter um GC é padrão de mercado?



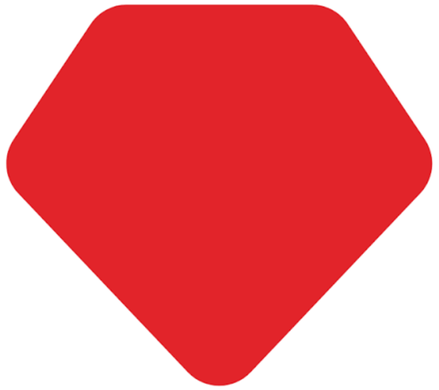




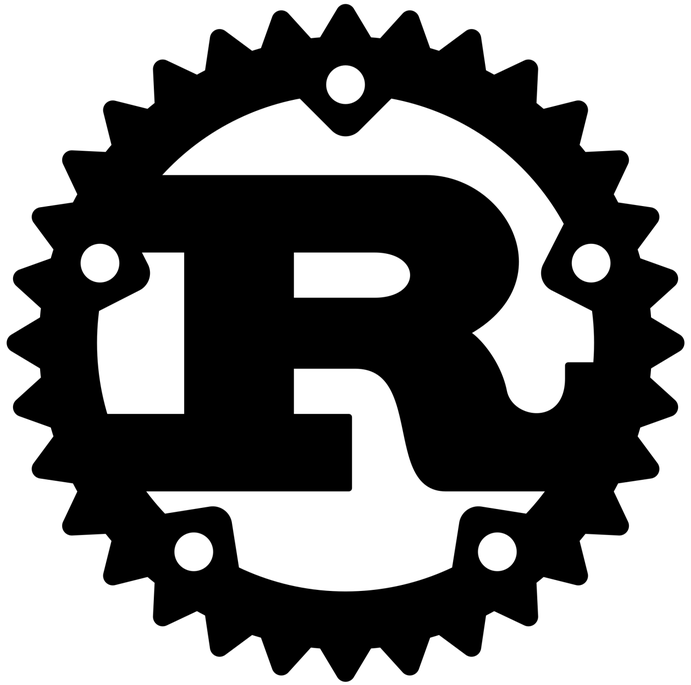
Sim…













e não!





Desempenho <> Simplicidade



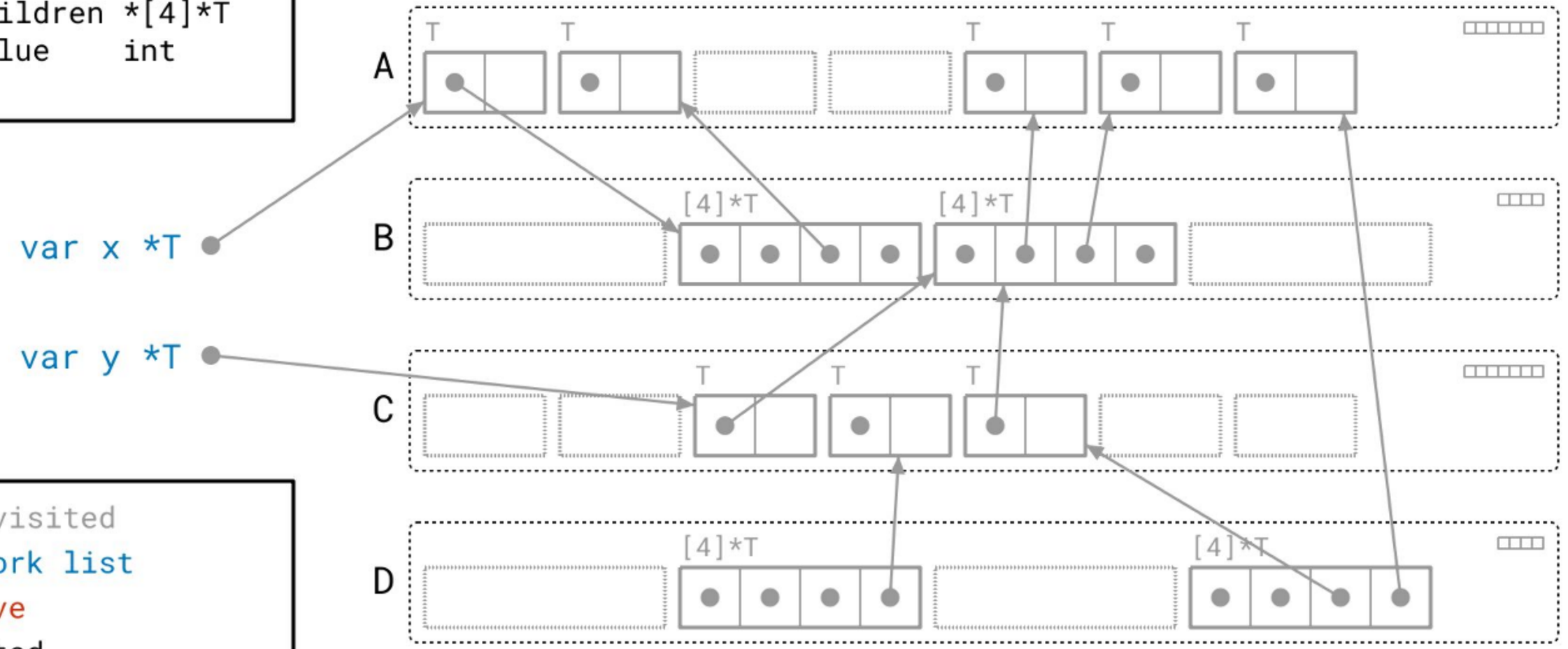
Como funciona o GC do Go?



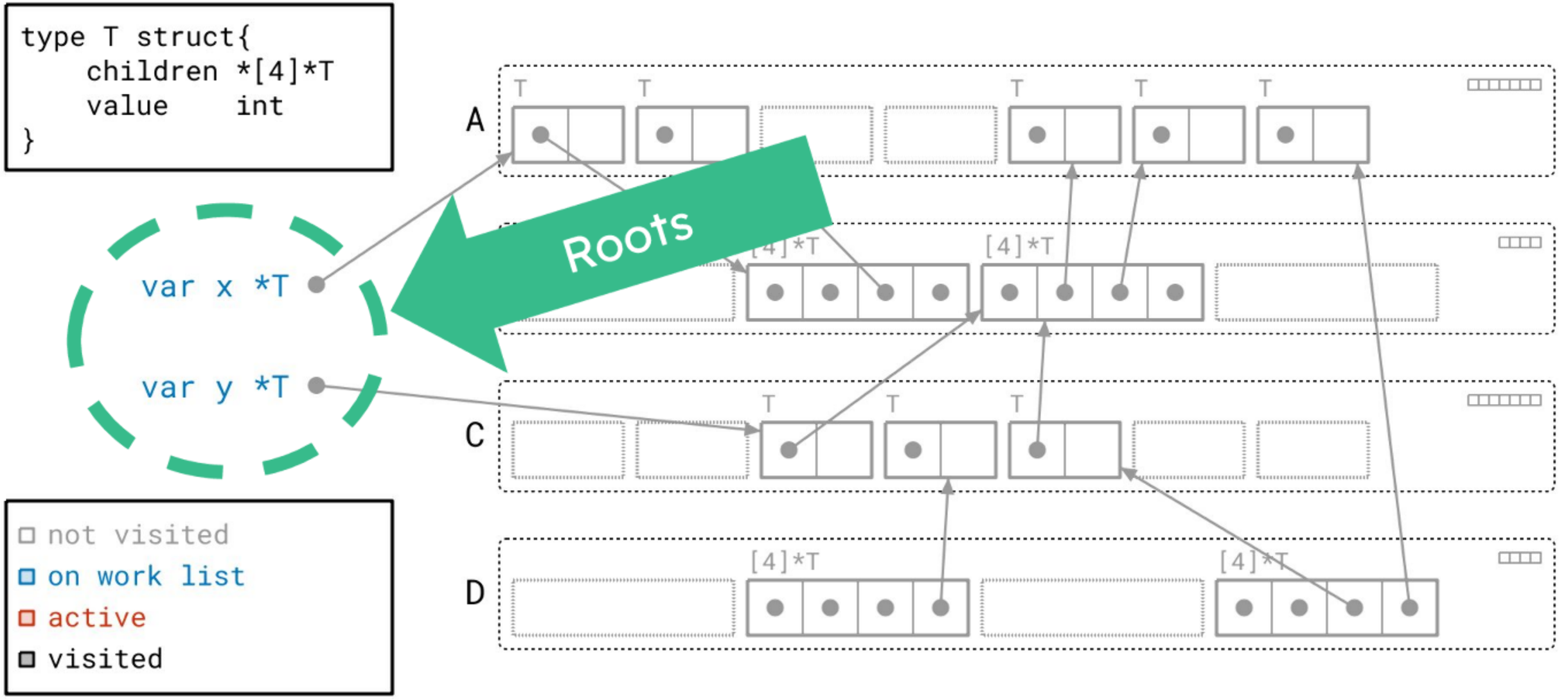
Go <= 1.25

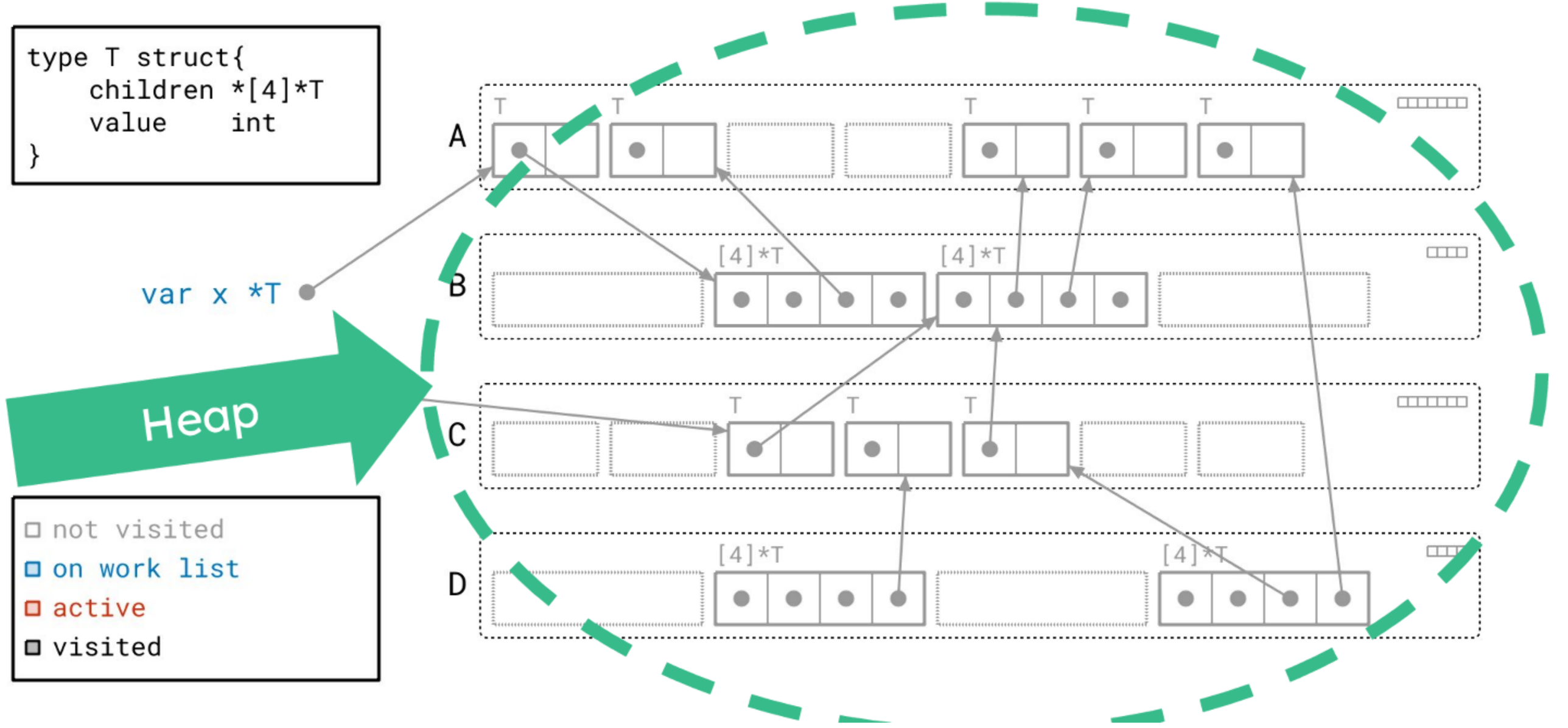


```
type T struct{
    children *[4]*T
    value    int
}
```

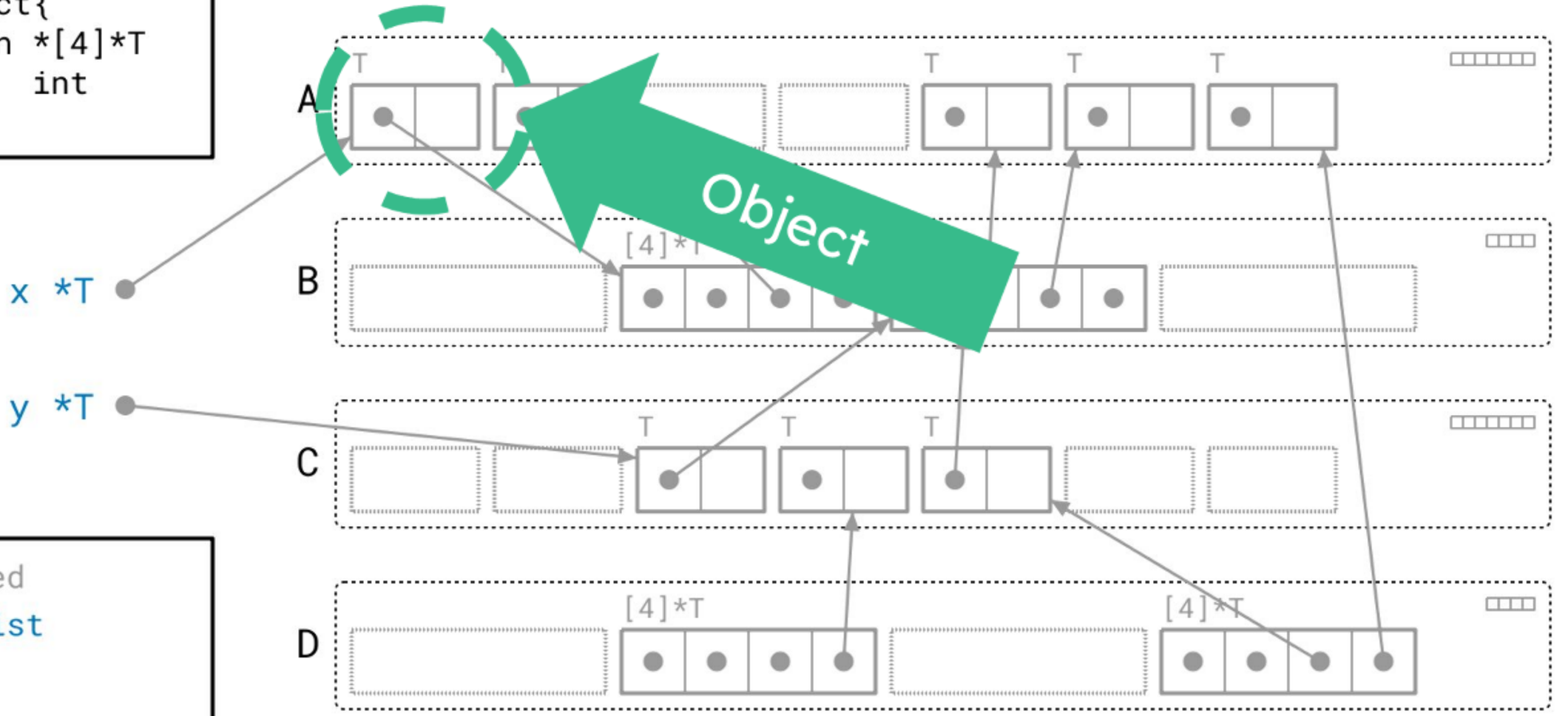


- not visited
- on work list
- active
- visited

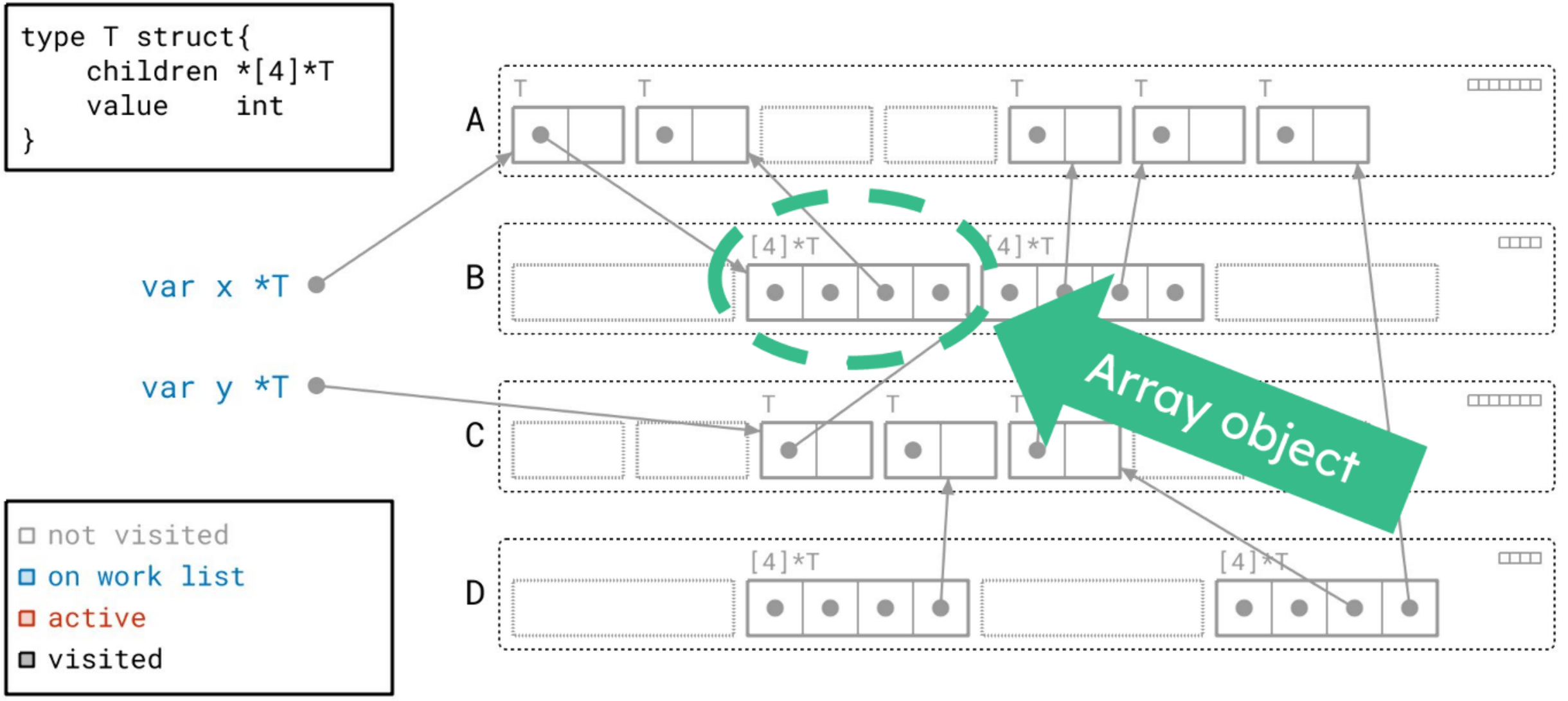

type T struct{
children *[4]*T
value int
}
var x *T
var y *T
Roots
A
C
D
T
[4]*T
not visited
on work list
active
visited

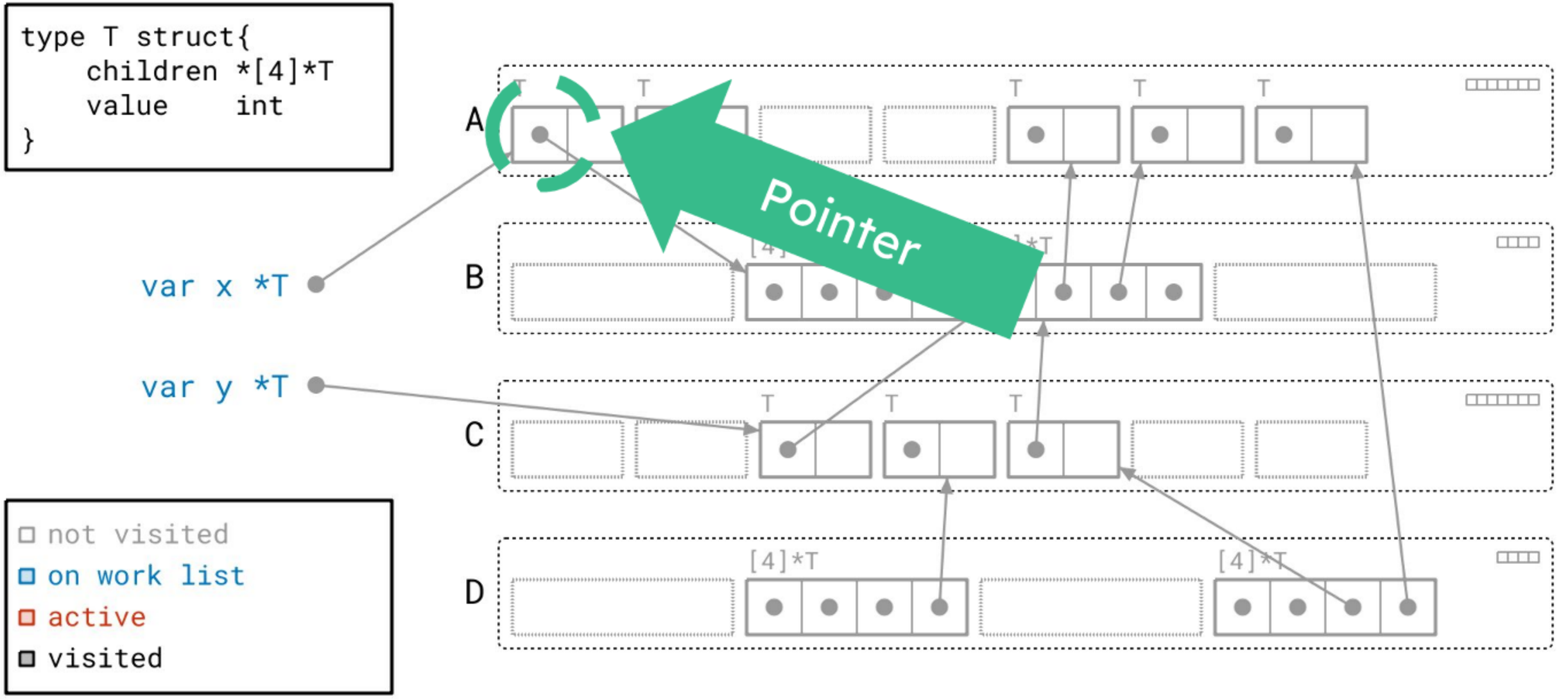

type T struct{
children *[4]*T
value int
}
var x *T
Heap
not visited
on work list
active
visited
A
B
C
D
T
[4]*T

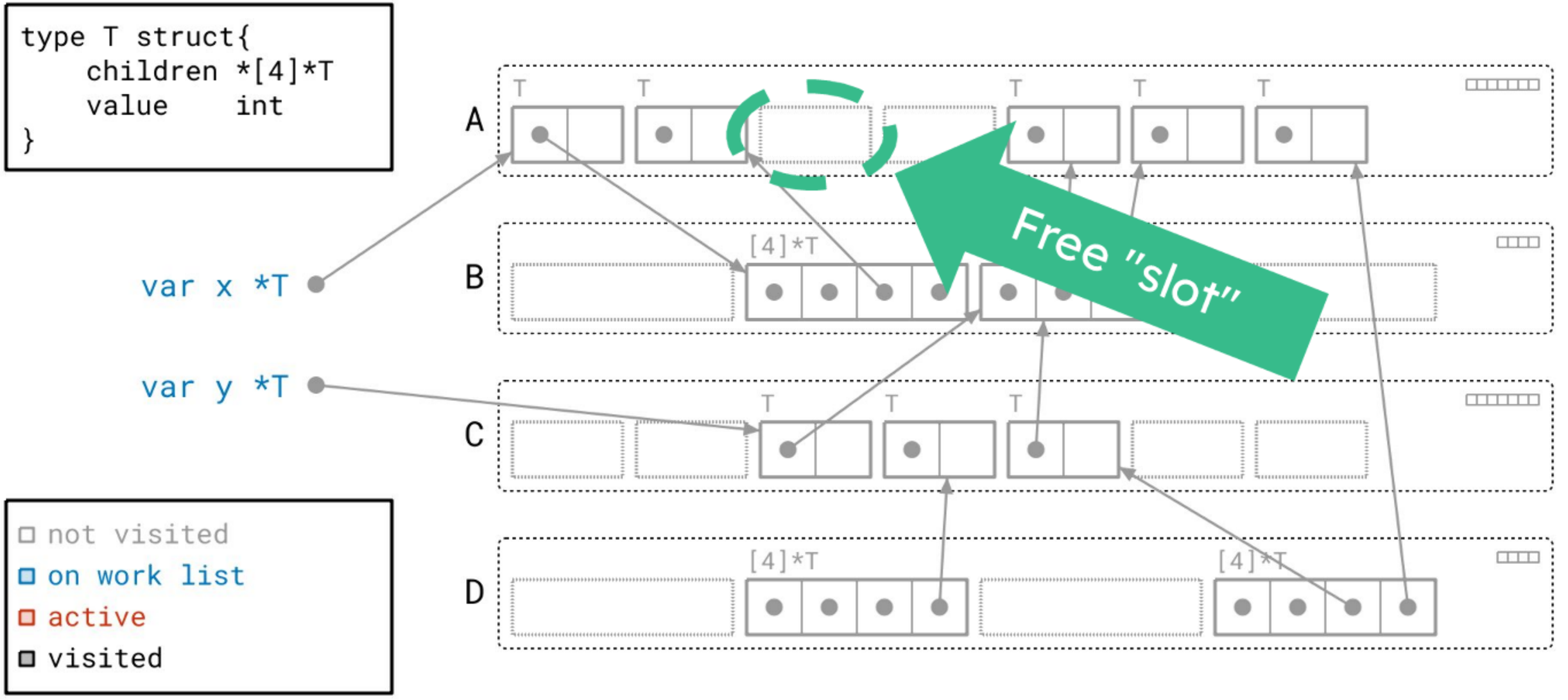
```
type T struct{
    children *[4]*T
    value    int
}
```

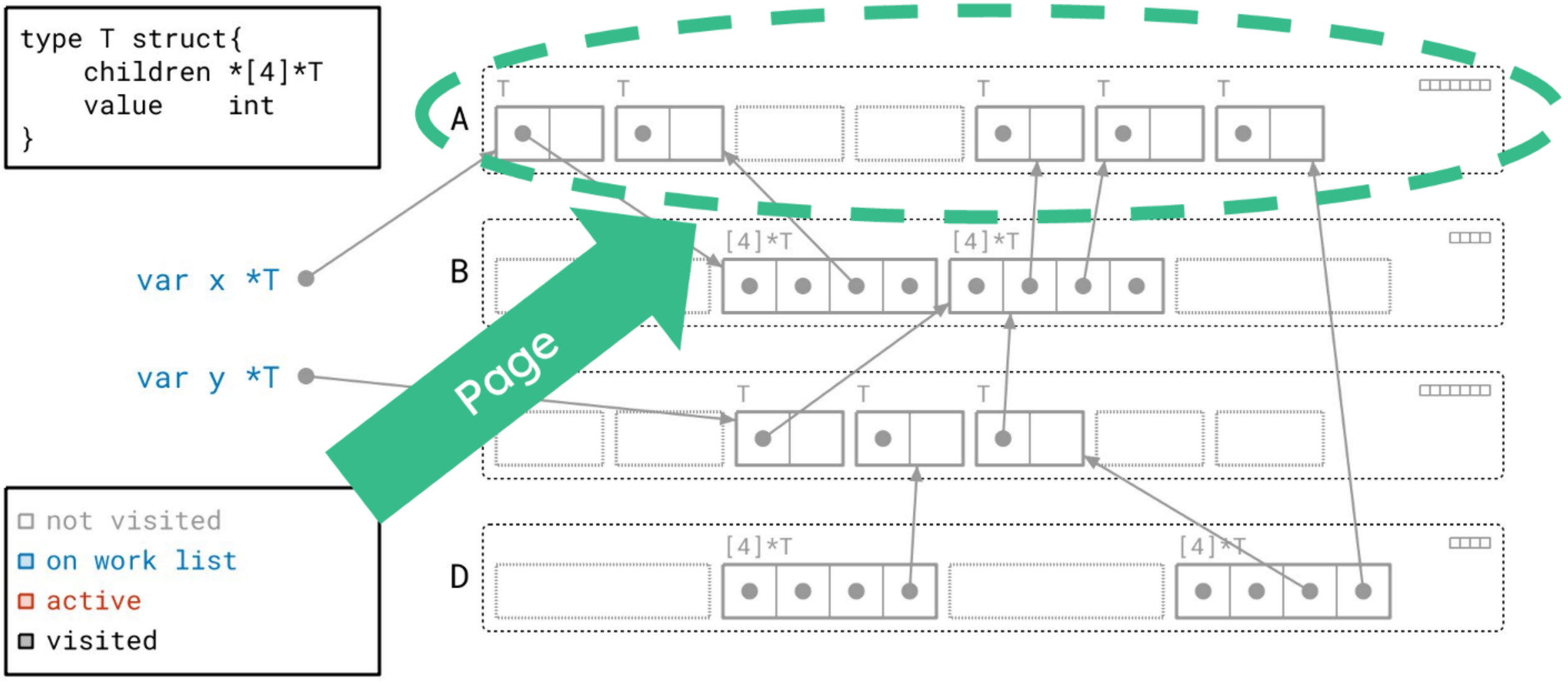


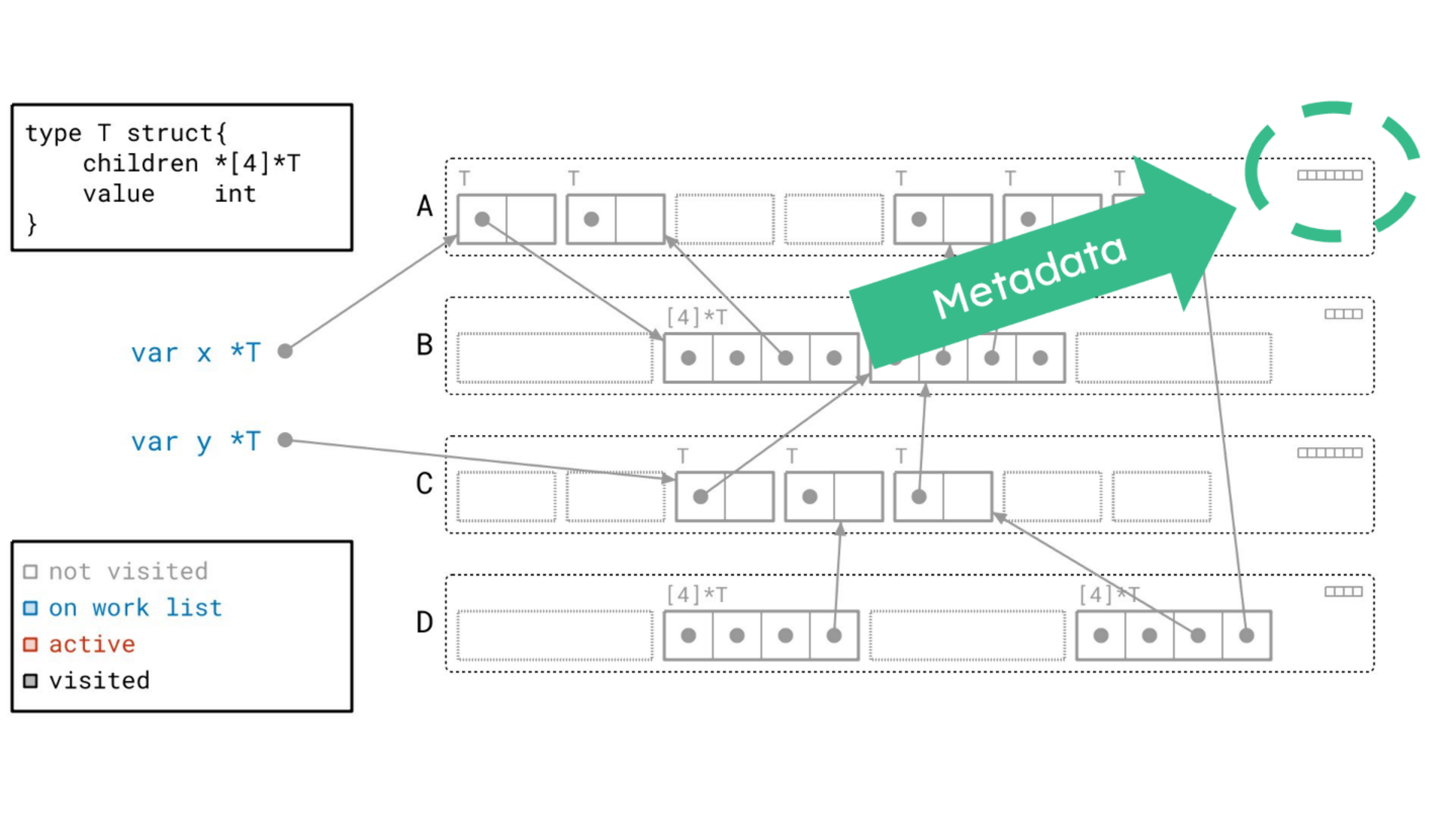
- not visited
- on work list
- active
- visited

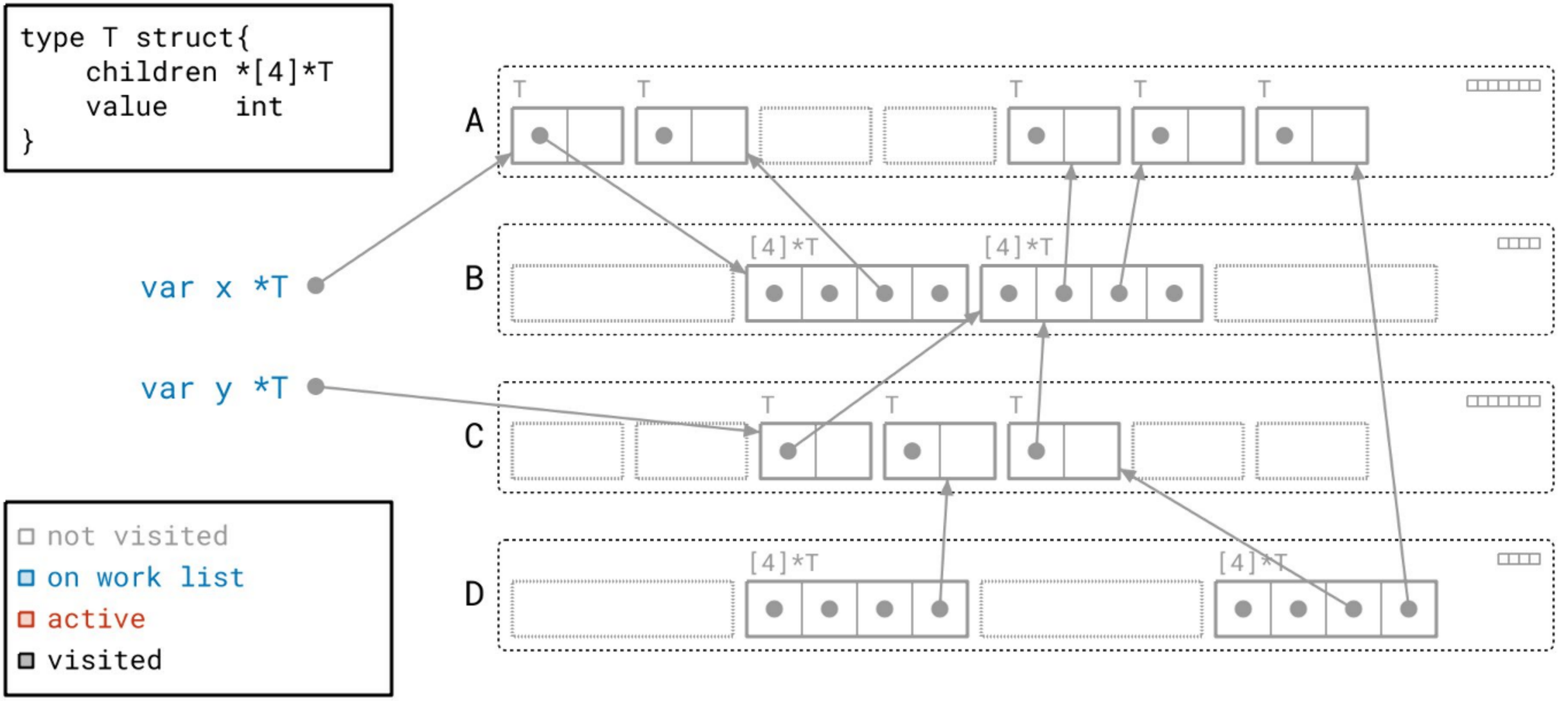

type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
Array object
not visited
on work list
active
visited


type T struct{
children *[4]*T
value int
}
A
B
C
D
Pointer
var x *T
var y *T
T
[4]*T
not visited
on work list
active
visited

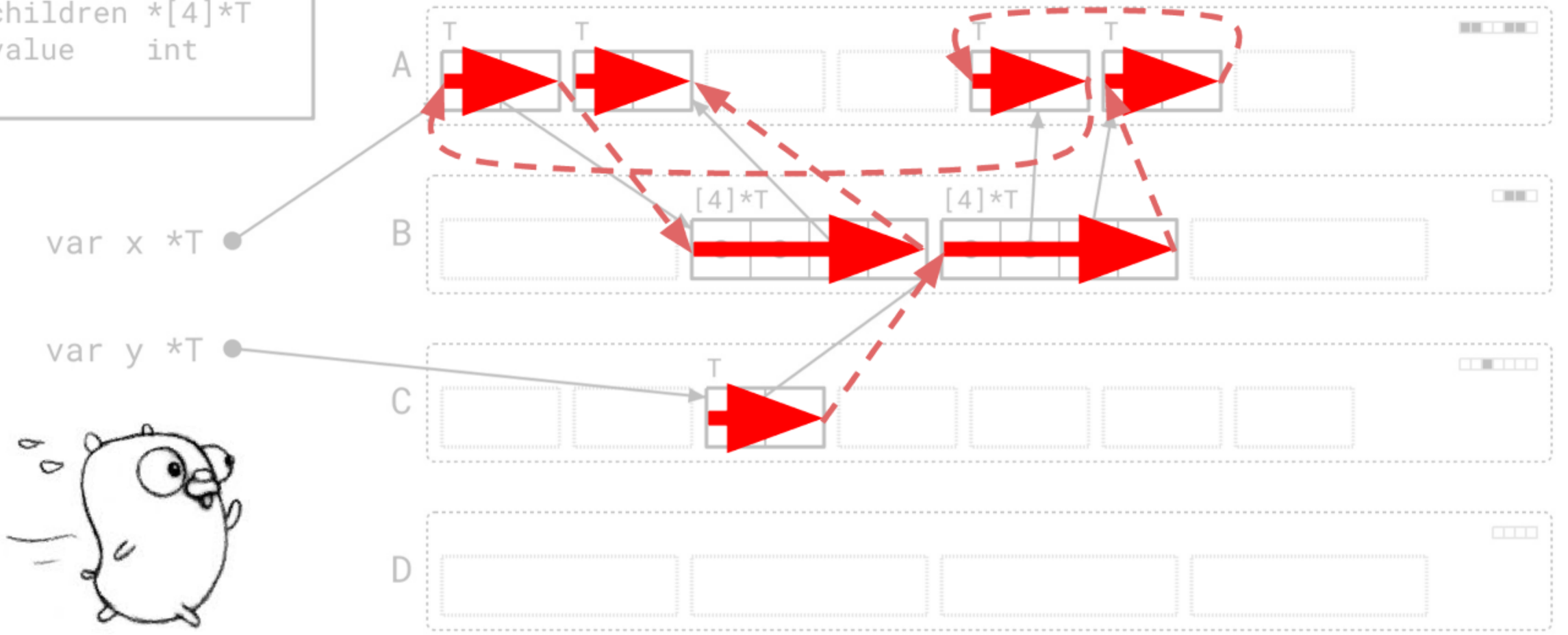

type T struct{
children *[4]*T
value int
}
var x *T
var y *T
not visited
on work list
active
visited
A
B
C
D
T
[4]*T
Free "slot"


type T struct{
children *[4]*T
value int
}
A
B
D
T
[4]*T
var x *T
var y *T
Page
not visited
on work list
active
visited


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
Metadata
not visited
on work list
active
visited


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

```
type T struct{
    children *[4]*T
    value    int
}
```



Certo, mas qual é o problema com esse algoritmo?



Total GC Cost =
cycles x average cost



“It’s not that uncommon to see Go programs spending 20% or more of their CPU time in the garbage collector.”

– go.dev



- “A microarchitectural disaster”
  - Acesso irregular a memória - NUMA
  - Redução de banda de memória
  - Mais cores de CPU
  - Hardware modernos

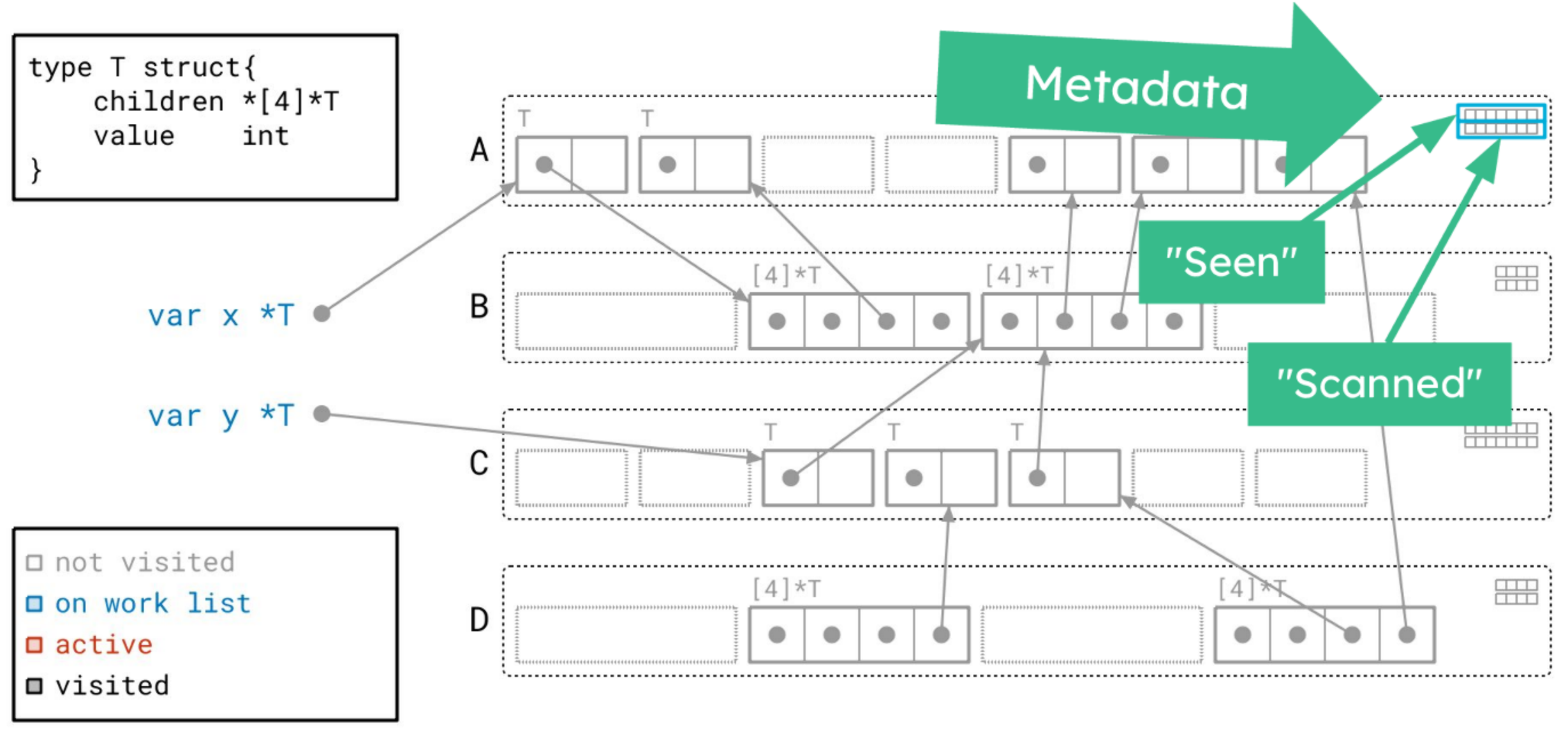


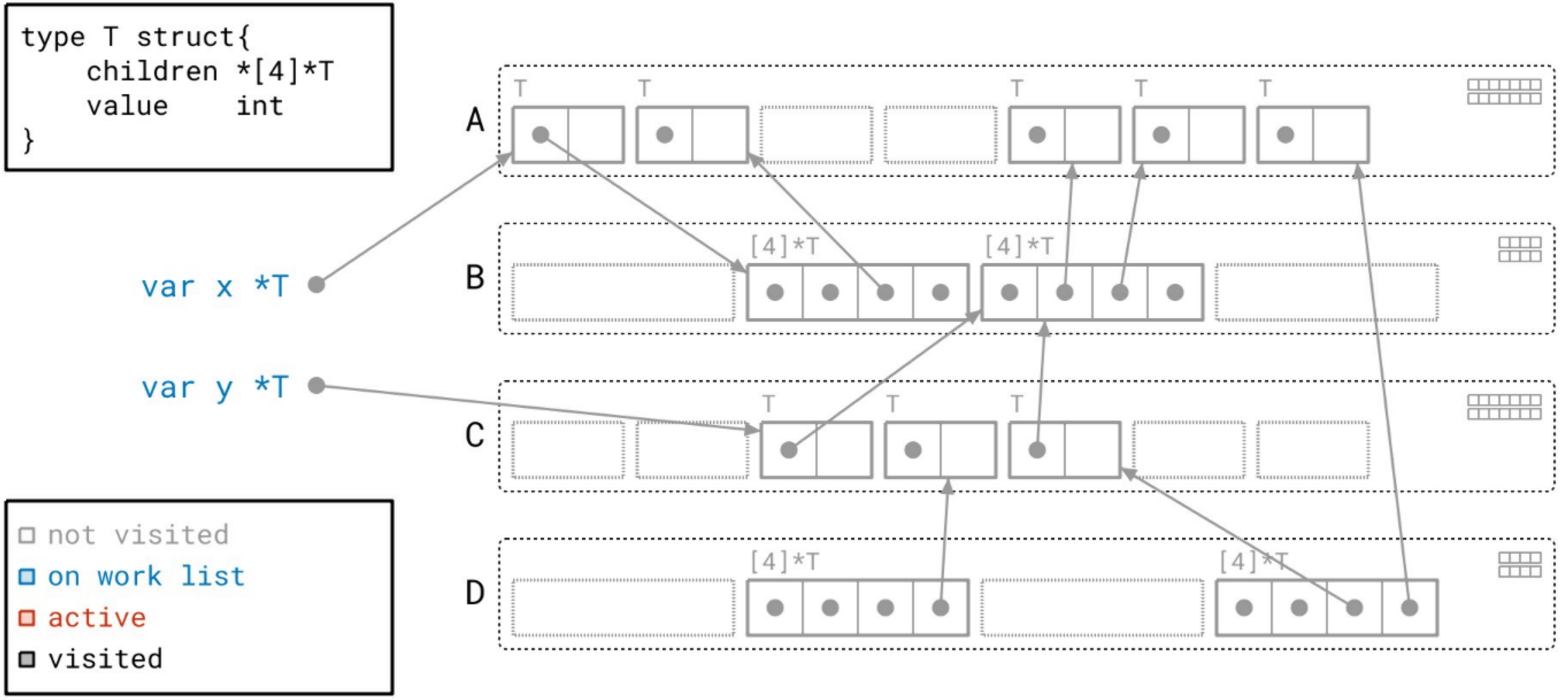
Vamos então destilar o chá-verde…



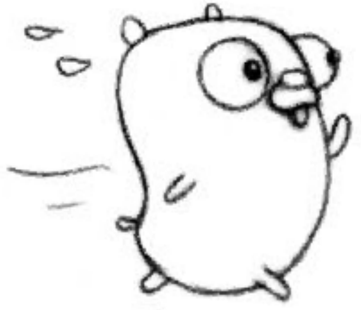
E se a gente olhasse para uma página inteira de memória?

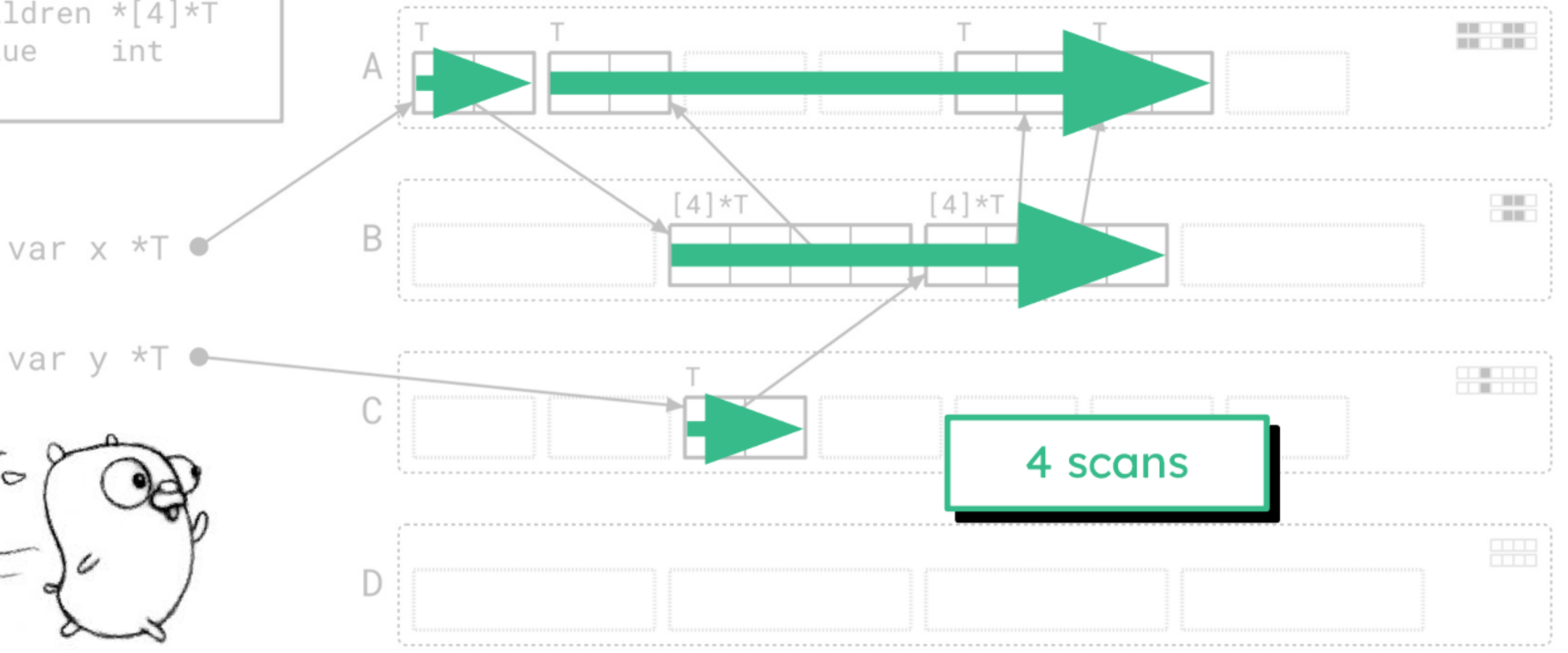



type T struct{
children *[4]*T
value int
}
not visited
on work list
active
visited
var x *T
var y *T
A
B
C
D
T
[4]*T
Metadata
"Seen"
"Scanned"


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

```
type T struct{
    children *[4]*T
    value    int
}
```





10% ~ 40% de redução no tempo de CPU



+10% de redução no tempo de CPU com Vector Acceleration



Mas nem sempre o chá é doce...



- Single object scans ou alocações dispersas
- Workloads com objetos muito grandes
- Tempo de GC é insignificante
- Outros gargalos



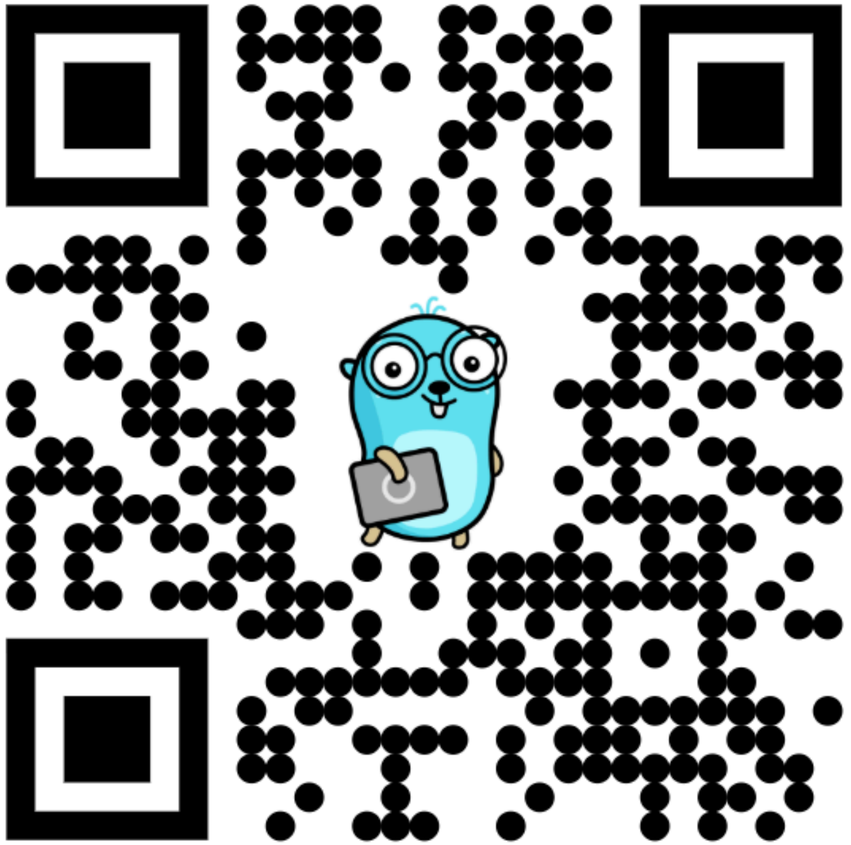
Green tea não é um algoritmo,
mas sim um design.



Links úteis

- https://go.dev/doc/gc-guide
- https://go.dev/blog/greenteagc
- https://alexrios.me/blog/greentea-gc-series-insight/
- https://www.dolthub.com/blog/2025-09-26-greentea-gc-with-dolt/





Obrigado!

Perguntas?

